Pràctica 1. Triggers

Sistemes de Gestió de Bases de Dades

Grau en Enginyeria Informàtica



DEPARTAMENT D'INFORMATICA
UNIVERSITAT DE VALÈNCIA

Curs 2026 / 2027

Autors: Ramón Cirilo / Vicente Cerverón

Pràctica 1. Triggers

Sistemes de Gestió de Bases de Dades

INDEX DEL DOCUMENT

Contingut

PART I. INTRODUCCIÓ ALS TRIGGERS 2
DISPARADORS O TRIGGERS 2
FUNCIONAMENT DELS TRIGGERS 2
CREACIÓ DE TRIGGERS 3
EXEMPLE DE TRIGGER 5
GESTIÓ D'ERRORS 6
EL PROBLEMA DE LES TAULES MUTANTS 7
GESTIÓ DELS TRIGGERS EN LA BASE DE DADES 8
SINTAXI COMPLETA DE LA SENTÈNCIA CREATE TRIGGER 10
BIBLIOGRAFÍA 11
PART II. DESENVOLUPAMENT DE LA PRÀCTICA 12
IMPLEMENTACIÓ DE TRIGGERS EN ORACLE 12
TAULES BASE 12
EXERCICIS 13
PRESENTACIÓ I EVALUACIÓ DEL TREBALL REALITZAT 14

Part I. Introducció als triggers

En moltes ocasions, les restriccions d'integritat que proporcionen els Sistemes de Gestió de Bases de dades no són suficients per a complir amb les regles de negoci establides en els requisits de les aplicacions que usen les dades emmagatzemades en aquests SGBD. La implementació d'aquestes regles de negoci es pot realitzar de dues maneres: a través del seu encapsulament en la codificació de l'aplicació, o a través de mecanismes procedurals dins de la base de dades (i per tant independents de l'aplicació).

La primera de les solucions és la menys recomanable, ja que suposa la creació d'un codi més difícil de mantenir, encara que el rendiment de l'aplicació puga ser millor que en la segona solució. La segona solució, per contra, és més recomanable a causa de la independència que aporta entre la capa de dades i la capa d'aplicació, però per a poder-la posar en pràctica és necessari que el SGBD dispose d'un gestor procedural que admeta en el seu llenguatge la supervisió d'esdeveniments sobre les dades emmagatzemades, i garantir que el rendiment en les operacions DML controlades pels programes de supervisió no es veu afectat a l'excés pel propi procés de supervisió.

Els procediments que permeten aplicar el control de la integritat de les dades segons el procés que s'ha descrit es diuen disparadors (o triggers en anglès). Aquests procediments s'emmagatzemen en la base de dades, juntament amb les dades que supervisen. Els triggers no solament permeten la creació de regles d'integritat complexes, o implementar regles de negoci, sinó que també permeten, per exemple, realitzar canvis en la base de dades de forma transparent per a l'usuari, calcular columnes derivades segons les dades que varien, mantenir la base de dades sincronitzada amb altres fonts de dades, permetre la inserció de registres en vistes compostes (no inseribles o modificables), auditar les operacions realitzades en la base de dades, controlar l'estructura de la base dades (no solament de les dades), etc.

La majoria dels SGBD comercials incorporen el seu propi llenguatge procedural que permet la creació dels triggers. En aquesta pràctica utilitzarem el SGBD de Oracle, i per tant s'adaptarà la definició, el concepte i la sintaxi dels operadors a aquest gestor, sent PL/SQL el llenguatge procedural a utilitzar.

DISPARADORS O TRIGGERS

Un trigger és una unitat PL/*SQL identificada (amb nom), que s'emmagatzema en la bases de dades, i que s'executa en resposta a un determinat esdeveniment que ocorre en la mateixa. Els triggers s'hauran de definir sobre les taules, vistes, esquemes o base de dades, segons corresponga.

Funcionament dels triggers

Un trigger respon a una manera de treball definit pel model Esdeveniment-Condició-Acció (ECA). Bàsicament, el disparador respondrà a un esdeveniment concret que es produïsca en la base de dades, comprovarà si es compleixen unes determinades condicions per a la seua activació i en cas que així siga, realitzarà una sèrie d'accions sobre la base de dades.

En el cas de Oracle, existeixen diversos tipus d'esdeveniments, associats a una taula, una vista, un esquema, i fins i tot la pròpia base de dades:

- Una sentència DML (DELETE, INSERT o UPDATE [OF column])
- Una sentència DDL (CREATE, ALTER o DROP)
- Una operació sobre la base de dades (SERVERERROR, LOGON, LOGOFF, STARTUP o SHUTDOWN)

En la pràctica que ens ocupa, ens centrarem exclusivament en esdeveniments relacionats amb sentències de tipus DML. Per a aquestes, l'acció, al seu torn, conté 3 aspectes indispensables per a una correcta definició:

- Les pròpies accions a realitzar, que es codifiquen en forma de codi PL/*SQL
- El timing, o el moment en el que es deuen executar les accions respecte a la ocurrència de l'esdeveniment. En ORACLE existeixen tres possibilitats:
  - BEFORE. S'executen les accions abans de que ocórrega l'esdeveniment.
  - AFTER. S'executen les accions després de que ocórrega l'esdeveniment.
  - INSTEAD OF. S'executen unes accions alternatives a les que s'especifiquen en l'esdeveniment que activa el trigger.
- L'àmbit d'actuació, que en ORACLE pot ser:
  - A nivell de sentència (opció per defecte). S'executa fins i tot quan l'esdeveniment no afecte a cap registre de la base de dades.
  - A nivell de registre: FOR EACH ROW. S'executa una vegada per a cada registre al que afecta l'esdeveniment, i no s'executa si no existeixen registres involucrats en l'esdeveniment.

Després de l'execució del disparador, el control torna al SGBD en el punt immediatament posterior a l'esdeveniment que ho va activar.

Creació de triggers

La sentència SQL que permet crear triggers té la següent estructura:

```
CREATE [OR REPLACE] TRIGGER trigger_name
   timing
   event1 [OR event2 OR event3]
ON object_name
[[REFERENCING OLD AS old | NEW AS new]
   [FOR EACH ROW]
   [WHEN (condition)]]
trigger_body
```

La sentència usa, com en molts altres casos de creació d'objectes, la paraula CREATE. En cas que ja existisca el disparador, el gestor retornarà un error. La clàusula OR REPLACE permet tornar a crear un disparador que ja existeix, reemplaçant-lo amb una nova definició.

La clàusula *timing* estableix el moment en el qual es va a executar el disparador. Aquesta clàusula, com ja s'ha vist, permet tres possibilitats: BEFORE | AFTER | INSTEAD OF. La primera executa el disparador abans d'executar l'esdeveniment que ho activa. La segona ho executa després, i la tercera evita l'execució de l'esdeveniment, substituint l'acció de l'esdeveniment pel que indica el disparador.

La clàusula d'esdeveniments permet especificar fins a 3 tipus d'esdeveniments diferents combinats amb l'operador OR. Les opcions possibles són: INSERT | DELETE | UPDATE [OF column]. Òbviament, es relacionen directament amb la clàusula ON object_name, ja que fan referència a operacions que es realitzen sobre una taula en qüestió, especificada per aquesta última clàusula.

La clàusula FOR EACH ROW és opcional, de manera que quan s'incloga s'executaran les accions per a cada registre modificat, i si no s'inclou s'executaran les accions una sola vegada, fins i tot si no hi ha registres afectats. Habitualment, als triggers que actuen a nivell de fila se'ls crida triggers-fila, i als quals actuen una única vegada se'ls crida triggers-sentencia.

És possible que perquè s'execute un disparador-fila s'hagen de satisfer les condicions que s'especifiquen en la clàusula WHEN, que és opcional. És important saber que aquesta clàusula no es pot usar en triggers-sentencia ni en triggers de tipus INSTEAD OF. L'expressió utilitzada en la clàusula s'avalua per a cada registre processat. Oracle no permet l'ús de funcions o procediments PL/SQL dins de l'expressió condicional.

Finalment, la secció trigger_body correspon al codi PL/SQL que es va a executar en resposta a l'esdeveniment i en el moment d'execució específic que s'haja programat. En aquest punt es pot optar per 2 opcions: a) codificar el programa en format PL/*SQL, o b) cridar a un procediment emmagatzemat a través de la funció CALL (exemple: CALL check_sal(:new.job_aneu, :new.salary, :new.last_name) cridaria a un procediment anomenat check_sal amb tres paràmetres), encara que aquest segon cas té més limitacions que el primer.

Els triggers-fila se solen usar per a controlar les accions sobre la base dels valors de la sentència d'inserció, modificació o esborrat. Per a accedir als valors que utilitza la sentència activadora i que afecten a la fila en qüestió, s'usen les variables :NEW i :OLD. El valor de :NEW fa referència al nou valor que quedarà en la fila (en tots els camps) després de l'execució de la sentència SQL. El valor de :OLD fa referència a l'antic valor que contenia la fila que està sent processada. La manera d'accedir a cada camp concret de la fila és amb la manera estàndard de referenciació de registre.camp (per exemple, :NEW.emp_id). Cal tenir en compte, a més, que en les insercions no es pot usar :OLD, en els esborrats no es pot usar :NEW, i només es poden usar tots dos en operacions d'actualització, per motius evidents.

En el cas que les expressions :NEW i :OLD no siguen del grat del programador, és possible modificar els seus noms a través de les clàusula REFERENCING. En ella serà necessari proporcionar els noms per a les variables corresponents als registres equivalents a :NEW i :OLD, i de nou açò només és aplicable al cas dels triggers-fila.

En l'acció del disparador, es pot saber quin és l'esdeveniment que ha causat la seua activació en el cas que s'hagen usat diversos esdeveniments en la definició. En aquest cas, podem usar sentències condicionals usant com a expressió condicional les següents:

- INSERTING és vertader si el trigger ha sigut activat per una sentència INSERT.
- DELETING és vertader si el trigger ha sigut activat per una sentència DELETE.
- UPDATING és vertader si el trigger ha sigut activat per una sentència UPDATE.
- UPDATING(col) és vertader si el trigger ha sigut activat per una sentència UPDATE que actualitza la columna col.

Per exemple:

```
CREATE OR REPLACE TRIGGER VALIDAR_DEPARTAMENTO
   AFTER INSERT OR DELETE OR UPDATE ON DEPARTAMENTO
   FOR EACH ROW
BEGIN
   IF INSERTING THEN
        ...
   END IF;
   IF UPDATING THEN
        ...
   END IF;
   IF DELETING THEN
        ...
   END IF;
END;
```

És important tenir en compte que no es poden usar els comandaments COMMIT, SAVEPOINT i ROLLBACK dins del cos d'un trigger.

Exemple de trigger

A continuació, i a manera orientativa, s'ofereix un exemple de creació d'un trigger:

```
CREATE OR REPLACE TRIGGER Print_salary_changes
  BEFORE DELETE OR INSERT OR UPDATE ON Emp_tab
  FOR EACH ROW
WHEN (new.Empno > 0)
DECLARE
    sal_diff number;
BEGIN
    sal_diff  := :new.sal  - :old.sal;
    dbms_output.put('Old salary: ' || :old.sal);
    dbms_output.put('  New salary: ' || :new.sal);
    dbms_output.put_line('  Difference ' || sal_diff);
END;
/
```

Gestió d'errors

En PL/SQL, a una condició d'error se li crida excepció (EXCEPTION). Quan es produeix una excepció, es pot especificar el seu tractament, és a dir, la forma en la qual s'ha de gestionar aquest error. Les excepcions que es poden manejar en PL/SQL són de dos tipus: les internes (pròpies de Oracle per a errors comuns, com a divisions per zero, falta de memòria, error en lectura de dades d'un cursor, etc.) i les definides per l'usuari (per a la gestió d'errors específics relacionats amb la lògica de negoci).

Les excepcions definides per l'usuari es poden declarar en la secció de declaració del procediment i es poden gestionar des de la secció EXCEPTION. A més, quan es detecta una condició d'error, es pot llançar la gestió de l'excepció a través de la sentència RAISE. Atès que un TRIGGER és un procediment PL/SQL, aquest principi també és aplicable als triggers. Un exemple de definició i de gestió d'excepcions definides per l'usuari pot ser el següent:

```
CREATE OR REPLACE PROCEDURE mi_procedimiento AS
    ...
    mi_excepcion EXCEPTION;
    ...
BEGIN
    ...
    IF (<condición de error>) THEN
        RAISE mi_excepcion;
    END IF;
    ...
EXCEPTION
    WHEN mi_excepcion THEN
        <gestión de la excepción>;
END;
/
```

Les excepcions representen una finalització del procediment de forma normal, mantenint les condicions de la transacció en la qual s'embeu l'execució del procediment. Per tant, quan s'apliquen a un trigger amb la intenció d'impedir que un procés es complete, per exemple, per falta d'integritat de les dades, és necessari complementar la gestió de l'excepció amb un tractament específic d'error d'aplicació (APPLICATION_ERROR). Aquesta tasca es pot realitzar de dues formes: es pot definir una excepció i llançar l'error d'aplicació quan es gestione, o bé llançar l'error d'aplicació directament quan es produeix l'error. La sentència que permet llançar l'error d'aplicació és RAISE_APPLICATION_ERROR, que consisteix en un procediment el funcionament del qual s'explica a continuació.

El procediment RAISE_APPLICATION_ERROR permet generar missatges d'error del tipus "ORA-" definits per l'usuari des de qualsevol procediment, funció o disparador. D'aquesta manera, qualsevol error que es genere durant la seua execució, pot ser capturat pel programa de nivell superior que estiga usant algun d'aquests procediments i tractar-ho convenientment.

La sintaxi d'aquest procediment és la següent:

```
raise_application_error(
      error_number, message[, {TRUE | FALSE}]);
```

on error_number és un enter negatiu que ha d'estar en el rang [-20999,-20000] i message és la cadena de caràcters que es retorna com a descripció de l'error. El tercer paràmetre és opcional i especifica si l'error s'ha de propagar (TRUE) o ha de substituir als errors previs (FALSE, opció per defecte).

Quan es llança un error d'aplicació, la transacció es deté en aqueix punt i queda a la discreció del programa de nivell superior que ha invocat el procediment prendre la decisió de què fer amb ella. Per exemple, si un trigger impedeix que s'inserisca un registre en una taula a través d'un error definit per l'usuari d'aquesta forma, la sentència INSERT retornarà el codi i missatge d'error corresponent, i el registre no haurà sigut inserit en la taula corresponent.

A continuació es mostra un exemple d'ús d'aquest procediment:

```
DECLARE
   num_tables NUMBER;
BEGIN
   SELECT COUNT(*) INTO num_tables FROM USER_TABLES;
   IF num_tables < 1000 THEN
      raise_application_error(-20101,
        'Expecting at least 1000 tables');
   ELSE
      NULL;
   END IF;
END;
/
```

El problema de les taules mutants

Una taula mutant és una taula que està sent modificada a través d'una operació de tipus DML (a nivell de contingut, no d'estructura). L'ús de triggers ha de contemplar la possibilitat que una taula estiga mutant quan s'executa, ja que en aqueix cas es generarà una excepció de tipus taula mutant i el disparador no acabarà la seua execució de forma normal i retornarà un error (per a garantir la consistència de les dades de la taula).

Les excepcions de taules mutants tenen lloc quan tractem de referenciar en un disparador la taula sobre la qual està definida el propi disparador, si es treballa a nivell de registre (no s'aplica en el cas de treballar a nivell de sentència). Bàsicament el problema succeeix quan s'accedeix al registre que està sent inserit, esborrat o modificat en la taula.

Per a il·lustrar com es produeix un error de taula mutant en Oracle podem analitzar el següent exemple. Suposem que vam crear el següent trigger, que conta tots els registres en la taula EMP després d'un esborrat:

```
CREATE OR REPLACE TRIGGER Emp_count
  AFTER DELETE ON emp
    FOR EACH ROW
DECLARE
  n INTEGER;
BEGIN
  SELECT COUNT(*) INTO n FROM emp;
  DBMS_OUTPUT.PUT_LINE('Hay ' || n || 'empleados.');
END;
/
```

Si s'executa ara la sentència:

```
DELETE FROM emp WHERE empno = 7499;
```

S'obtindrà el següent error:

```
ORA-04091: table HR.emp is mutating, trigger/function
might not see it
```

Es pot observar que la consulta de dades dins del trigger sobre la pròpia taula que gestiona el trigger genera un error de taula mutant que interromp la transacció.

Els errors de taules mutants es poden resoldre de diverses formes. En algunes ocasions, si és possible, n'hi ha prou amb canviar el *timing* del disparador, i usar BEFORE en lloc de AFTER. No obstant açò, aquesta opció no sempre és possible, com en l'exemple que hem vist.

Una altra possible solució consisteix a usar una taula temporal i construir dues triggers, un a nivell de registre que actua sobre la taula temporal i un altre a nivell de sentència que actua sobre la taula original. Aquesta opció és més complexa, i s'ha d'implementar amb molta precaució.

Gestió dels triggers en la base de dades

El diccionari de dades de ORACLE emmagatzema informació sobre tots els objectes que maneja (taules, vistes, etc.), i els triggers no són una excepció. És possible consultar el diccionari de dades per a conèixer els triggers que existeixen i les seues característiques.

Una de les primeres coses a tenir en compte és que els triggers en ORACLE poden estar en estat activat (ENABLED) o desactivat (DISABLED). Òbviament, un trigger només reaccionarà davant l'esdeveniment per al qual ha sigut programat si es troba activat. Aquesta informació es pot consultar en el diccionari de dades, i l'estat del trigger es pot modificar a través de sentències SQL.

Per a consultar els triggers cal usar la taula del diccionari de dades anomenada USER_TRIGGERS, que té aquesta estructura parcial (es pot consultar la definició completa en la BD usant DESC USER_TRIGGERS):

COLUMNA	DESCRIPCION
TRIGGER_NAME	Nom del trigger
TRIGGER_TYPE	El tipus és BEFORE, AFTER o INSTEAD OF
TRIGGERING_EVENT	Operació que activa el trigger
TABLE_NAME	Nom de la taua sobre la que actua
WHEN_CLAUSE	Condició d'execució del procediment de control
STATUS	Estat d'activació
TRIGGER_BODY	Codi del procediment que s'executa quan s'activa el trigger

Per tant, per a obtenir informació detallada d'un trigger, n'hi ha prou amb executar una consulta SQL que seleccione la informació desitjada de la taula USER_TRIGGERS i filtre per les condicions desitjades, per exemple:

```
SELECT trigger_name, trigger_type, triggering_event,
  table_name, referencing_names,
  status, trigger_body
FROM user_triggers
WHERE trigger_name = 'RESTRICT_SALARY';
```

Per a modificar l'estat d'un trigger, es pot treballar a nivell individual o de taula:

- A nivell individual, es pot activar o desactivar un determinat trigger:
  `ALTER TRIGGER trigger_name DISABLE | ENABLE;`
- A nivell de taula, es poden activar o desactivar tots els triggers d'una determinada taula:
  `ALTER TABLE table_name DISABLE | ENABLE ALL TRIGGERS;`

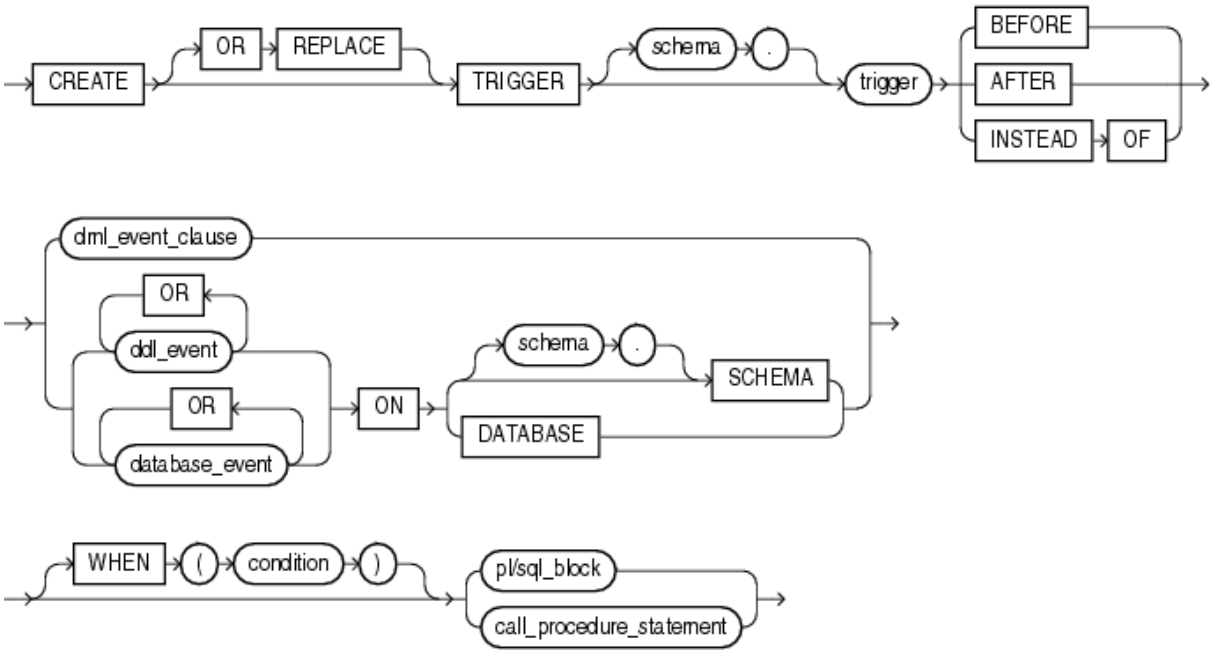
Per a eliminar un trigger, s'usa la sentència DROP:

```
DROP TRIGGER trigger_name;
```

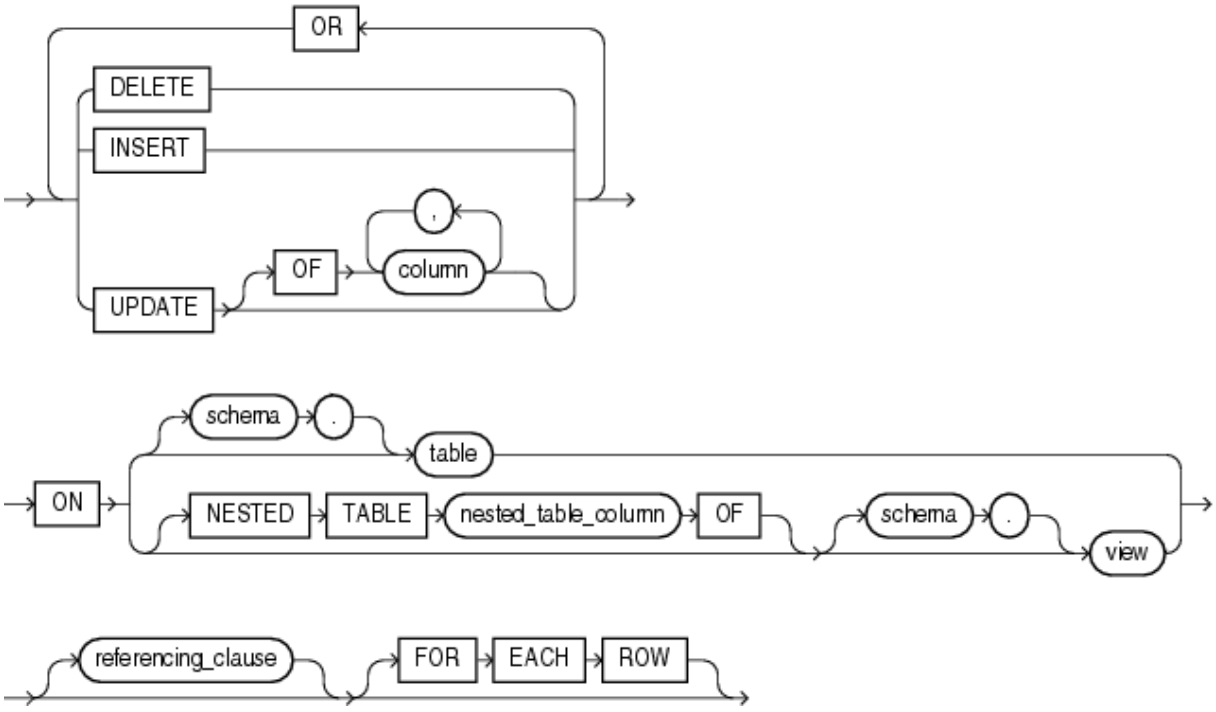
Sintaxi completa de la sentència CREATE TRIGGER

Segons el manual de referència de ORACLE, la sintaxi de la sentència CREATE TRIGGER és la següent:

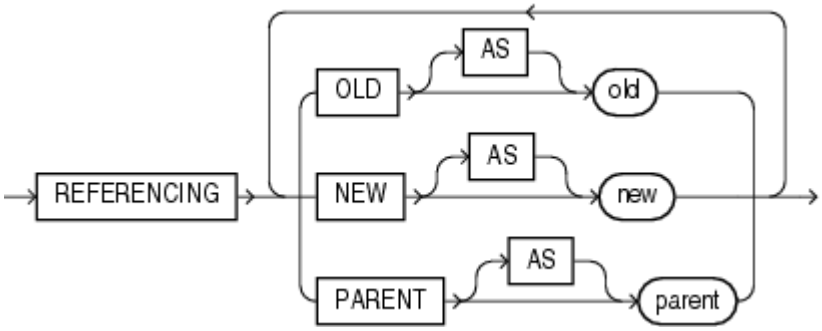
`create_trigger::=`



`DML_event_clause::=`



Referencing_clause::=



BIBLIOGRAFÍA

Sintaxi de la sentència CREATE TRIGGER.
https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/CREATE-TRIGGER.html

Us de triggers en Oracle.
https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/plsql-triggers.html

Oracle PL/SQL by example. Benjamin Rosenzweig, Elena Silvestrova Rakhimov. Addison-Wesley, 2009. ISBN: 9780137151684

Oracle PL/SQL programming. Steven Feuerstein, Bill Pribyl. O'Reilly Media, 2009. ISBN: 9780596805401

Part II. Desenvolupament de la pràctica

IMPLEMENTACIÓ DE TRIGGERS EN ORACLE

En aquesta pràctica s'utilitzarà la base de dades Oracle per a la implementació dels triggers, seguint els conceptes teòrics explicats amb anterioritat.

El desenvolupament es realitzarà sobre una sèrie de taules que els estudiants hauran de crear en el seu esquema i treballar exclusivament sobre elles.

L'accés a la base de dades Oracle es realitzarà utilitzant les credencials que proporcionarà el professor. L'eina de treball serà el programa SQL Developer, i la configuració de la connexió al servidor de base de dades serà:

- Servidor: pokemon.uv.es
- Identificador de Servei: ORCL
- Port: 1521

La pràctica té una durada de 2 sessions.

Taules base

En cada esquema s'hauran de crear dues taules perquè continguen informació sobre rius i països, ambdues relacionades a través de claus alienes per a conèixer els llocs on naixen i desemboquen. Les taules seguiran els següents esquemes lògics:

Taula: RIOS

COLUMNA	TIPO	PK	NN	UNI	CHK	FK	COMENTARI
ID_RIO	NUMBER	✓					
NOMBRE	VARCHAR2(100)		✓				
LONGITUD	NUMBER						
CUENCA	NUMBER						
CAUDAL_MEDIO	NUMBER						
NACIMIENTO	VARCHAR2(64)		✓				
PAIS_NACIMIENTO	NUMBER		✓			✓	PAISES(ID_PAIS)
DESEMBOCADURA	VARCHAR2(64)		✓				
PAIS_DESEMBOC	NUMBER		✓			✓	PAISES(ID_PAIS)

Taula: PAISES

COLUMNA	TIPO	PK	NN	UNI	CHK	FK	COMENTARI
ID_PAIS	NUMBER	✓					
NOMBRE	VARCHAR2(64)		✓				
CONTINENTE	VARCHAR2(24)		✓				
CAPITAL	VARCHAR2(64)		✓				
SITUACION	VARCHAR2(128)						
NOTAS	VARCHAR2(128)						

Exercicis

L'alumne haurà d'implementar una sèrie de triggers sobre les taules amb les següents característiques:

1. Crear les taules definides amb anterioritat i emplenar-les amb dades de prova.

2. Implementar un disparador PL/SQL que impedisca que s'inserisquen o modifiquen registres en la taula RIOS, assignant-los una longitud negativa o excessivament gran. Fer el mateix amb el cabal. Comprovar el funcionament del disparador realitzant les següents tasques:

    a) Contar els rius que hi ha a la taula RIOS al començament del procés.
    b) Inserir un riu amb una longitud permesa (INSERT) y consultar la taula per a comprovar que ha funcionat la inserció (SELECT)
    c) Modificar un riu amb una longitud no permesa (UPDATE) y mostrar l'error que genera el SGBD
    d) Deshabilitar el trigger (sentència SQL) creat
    e) Repetir l'operació de modificació i consultar la taula per a comprovar que la dada de longitud incorrecta s'ha inserit (SELECT)
    f) Habilitar el trigger (sentència SQL)
    g) Executar un ROLLBACK per a desfer la transacció.
    h) Contar els rius que hi ha a la taula RIOS després del ROLLBACK, comparar el numero amb l'obtingut al principi, i comentar el resultat obtingut.

3. Implementar un trigger PL/SQL que impedisca que s'inserisquen o modifiquen registres en la taula RIOS assignant-los un país de desembocadura que no estiga a Europa.

4. Incloure un nou camp en la taula PAISES anomenat NUM_RIOS per a guardar el nombre de rius que desemboquen en aqueix país, i implementar un trigger perquè mantinga actualitzat aquest camp NUM_RIOS en la taula PAISES quan la taula RIOS patisca qualsevol canvi que afecte a aquest valor

5. Implementar un o més triggers PL/SQL que mantinguen la taula CAMBIOLOG actualitzada amb les dates dels 10 últims canvis que s'hagen realitzat sobre les taules PAISES i RIOS. S'ha d'emmagatzemar al menys el nom de la taula modificada i la data de la modificació (usar SYSDATE). *Nota: S'haurà de definir i crear la taula CAMBIOLOG.*

6. Implementar un trigger que genere un problema de taula mutant i comprovar el seu funcionament amb una sentencia SQL.

7. Implementar un trigger per a cada taula que genere automàticament la clau primària sense necessitat de proporcionar explícitament un valor en el moment de crear un nou registre. S'haurà de generar el valor del camp només en el cas en que no es proporcione cap.

8. Suposem que s'han definit dos triggers del mateix tipus sobre una mateixa taula, i l'ordre d'execució dels triggers és important per a poder obtindré un resultat consistent. Investiga la forma en la que es podria definir en Oracle l'ordre d'execució de triggers sobre la mateixa taula i posa un exemple de dos triggers sobre la taula països, TRG1 i TRG2, de manera que s'execute TRG1 sempre abans que TRG2.

Presentació i evaluació del treball realitzat

Els alumnes hauran de pujar a l'aula virtual, al final de la segona sessió de la pràctica, un fitxer de script en format de text pla amb el codi SQL i PL/SQL necessari per a l'execució completa de les tasques encomanades en la pràctica.

El professor valorarà el treball dels estudiants en finalitzar la primera sessió de pràctiques, de manera que els alumnes hauran d'indicar el grau de desenvolupament de la pràctica.

El professor valorarà al final de la pràctica no solament el codi generat, sinó també la funcionalitat del codi implementat pels alumnes.